

ORTHO2CAD: 3D CAD GENERATION FROM ORTHOGRAPHIC DRAWINGS USING VISION LANGUAGE MODELS

Aditya Joglekar

Amit Regmi

Kenji Shimada

Levent Burak Kara*

Department of Mechanical Engineering
Carnegie Mellon University
Pittsburgh, PA, 15213, USA

ABSTRACT

Engineering design intent is often communicated through rasterized orthographic drawings. However, downstream workflows inherently require editable and parametrically defined 3D computer-aided design (CAD) models. To bridge this gap, we introduce Ortho2CAD, a vision-language model (VLM) specifically designed to translate rasterized orthographic drawings directly into editable CadQuery code, which can then be seamlessly converted into 3D CAD models. To train the model effectively, we utilize supervised fine-tuning (SFT) for instances where explicit CadQuery code labels already exist, and we apply geometry-grounded reinforcement learning (RL) to optimize the model in scenarios where ground-truth labels are absent. To enable learning at scale, we create a pythonOCC-based drawing generator that renders first-angle orthographic projections from STEP models, complete with dashed hidden lines and key dimensions. On existing datasets encompassing settings both with and without CadQuery supervision, we generate orthographic drawings and show that our model produces 100% syntactically valid code. Moreover, it achieves a 3D CAD intersection-over-union (IoU) accuracy that surpasses all baselines, with an average relative improvement of over 7% compared directly against the next best performing model. We show that leveraging VLMs with SFT and RL techniques can effectively pave the way forward for orthographic drawing to 3D CAD reconstruction. Our implementation is available at <https://github.com/AdityaJoglekar/Ortho2CAD>.

1 INTRODUCTION

3D computer-aided design (CAD) models are the dominant representation for mechanical parts and are directly usable in

downstream engineering workflows such as manufacturing planning, simulation and cost estimation. Yet, much of the design intent in practice is communicated through 2D engineering technical drawings, specifically multi-view orthographic that provide a standardized way to encode topology and dimensions. While these drawings may be authored digitally (often in vector form), the format most frequently exchanged in real manufacturing workflows is often an image-based (raster) drawing due to ease of sharing, quality assurance and intellectual-property protection because image drawings are not editable [1]. This prevalence of raster drawings creates a major impediment to 3D reconstruction automation because unlike vector formats that provide direct scripted access to geometric and semantic entities, raster drawings typically require human inspection to extract the information. Together, these realities create a persistent disconnect between how designs are specified and communicated (often as rasterized 2D drawings) and how they must be reconstructed as editable 3D CAD by designers and engineers for subsequent applications.

Recent progress in large language models (LLMs) and vision language models (VLMs) has enabled direct generation of CAD programs from text descriptions, perspective images and point clouds exemplified by works such as [2], [3] and [4]. They produce CadQuery¹ code and evaluate the resulting solids using geometric similarity metrics such as intersection-over-union (IoU). While these approaches demonstrate the CadQuery code-generating abilities of LLMs and VLMs, none of them tackle the task of converting 2D engineering drawings to CadQuery code. 2D engineering drawings with orthographic projections of a part are the standard for part design communication and a prime input modality designers deal with for 3D model creation. Hence, we focus on orthographic projection images as input to a VLM for

*Address all correspondences to lkara@andrew.cmu.edu

¹CadQuery is a Python based parametric CAD scripting library

generating CadQuery code and thus creating editable 3D CAD models.

Data synthesis. Due to the unavailability of large-scale engineering drawing data, we create a scalable data generation python code using the open source pythonOCC library to synthesize orthographic drawings with three views and critical bounding box dimension annotations from existing CAD repositories. We generate these drawings for the DeepCAD [5] and Fusion 360 Reconstruction [6] datasets and publish the data generation pipeline for further use by the community. Zhang et al. [7] release a 2D orthographic drawings dataset constructed from the Fusion 360 database and using FreeCAD [8], but it only consists of simple 2981 samples and we could not find their exact generation pipeline on any public platform. Zhang et al. [9] use the ABC dataset [10] and FreeCAD [8] for creating close to 70k 2D orthographic drawings but the drawings do not have key dimensions and do not have dashed hidden lines, thus not following the standard conventions. We present one of the largest open-source resources (more than 150k samples) of orthographic drawings, and importantly publish an automatic generation python code for orthographic drawings with dashed hidden lines and bounding box dimensions derived from public CAD datasets, enabling systematic study of drawing-to-editable-CAD generation under controlled, reproducible conditions.

With this data foundation, two complementary learning regimes are investigated: supervised finetuning and reinforcement learning, reflecting the practical reality that code supervision is uneven across datasets.

Supervised finetuning on DeepCAD dataset. On DeepCAD, where CAD-Coder [3] provides ground-truth CadQuery codes/programs (GenCAD-Code), supervised fine-tuning of Qwen3VL-8B-Instruct [11] is performed to translate orthographic drawings into CadQuery code. Compared to the CAD-Coder baseline, fine-tuning a stronger, newer VLM backbone on the same underlying objective improves reconstruction quality. It also indicates that orthographic drawings can be an effective conditioning signal for code synthesis when paired code supervision exists.

Reinforcement learning on Fusion 360 reconstruction dataset. Almost all CAD datasets, including Fusion360 reconstruction, do not provide aligned ground-truth CadQuery codes in a form directly usable for supervised code synthesis. This setting motivates reinforcement learning (RL) with geometric feedback, where the model can be optimized using reward signals computed from the CAD produced by executing generated code, rather than relying on target codes. We use a Dr. GRPO [12] style group-relative objective with sequence-level optimization and apply it to orthographic-drawing-to-3D-CAD reconstruction via CadQuery code on the Fusion360 dataset, showing that policy optimization driven by geometric similarity rewards substantially improves 3D reconstruction accuracy over supervised-only baselines and compensates for the absence of ground-truth Cad-

Query codes.

Overall, our approach (Figure 1) reframes editable CAD reconstruction using VLMs around an engineering native specification format of orthographic drawings and provides a pathway across both code supervision and unsupervised regimes. The key contributions are:

1. supervised VLM fine-tuning results on DeepCAD dataset demonstrating state-of-the-art drawing-to-CAD synthesis over prior baselines when paired CadQuery code is available.
2. an RL-based training strategy for datasets without ground-truth CadQuery codes, where optimization against executable-geometry rewards yields the highest output code validity and 3D reconstruction accuracy compared to all baselines.
3. an orthographic drawing dataset and python code for generating large-scale orthographic drawing datasets from CAD models.

2 LITERATURE REVIEW

Learning-based geometry generation with prediction of representations such as voxels, meshes and point clouds [13–16] has seen significant development over the past few years. For engineering tasks, there has also been a development in predicting Boundary representation (B-rep) solids [17, 18]. Downstream tasks benefit from feature-level editability and several recent methods highlighted below increasingly treat CAD modeling as synthesizing codes that enable editability and which can be executed to obtain B-rep solids.

CAD datasets and representations. A key enabler for CAD synthesis is the availability of large-scale CAD datasets, where having sequential operations for CAD models in the dataset can be very useful. The ABC dataset [10] is a large repository of 1 million complex CAD models. Fusion 360 Gallery dataset [6] provides a reconstruction dataset of 8,625 designs, comprising sequential sketch and extrude modeling operations. DeepCAD [5], which consists of a subset of models from the ABC dataset, introduces more than 150k CAD models paired with construction sequences and demonstrates that Transformer-style sequence models can learn distributions over CAD programs. CAD-Coder [3] extends this dataset and creates the GenCAD-Code dataset consisting of CAD-model image and CAD Query code pairs.

LLM-based CAD program synthesis. An end-to-end transformer-based autoregressive network is proposed by [19] to generate parametric CAD models from input texts. Guan et al. [2] reformulate text-to-CAD as the generation of CadQuery scripts. This representation enables a seamless integration with existing LLMs. Their approach proposes a two stage pipeline of supervised fine-tuning and then reinforcement learning with a

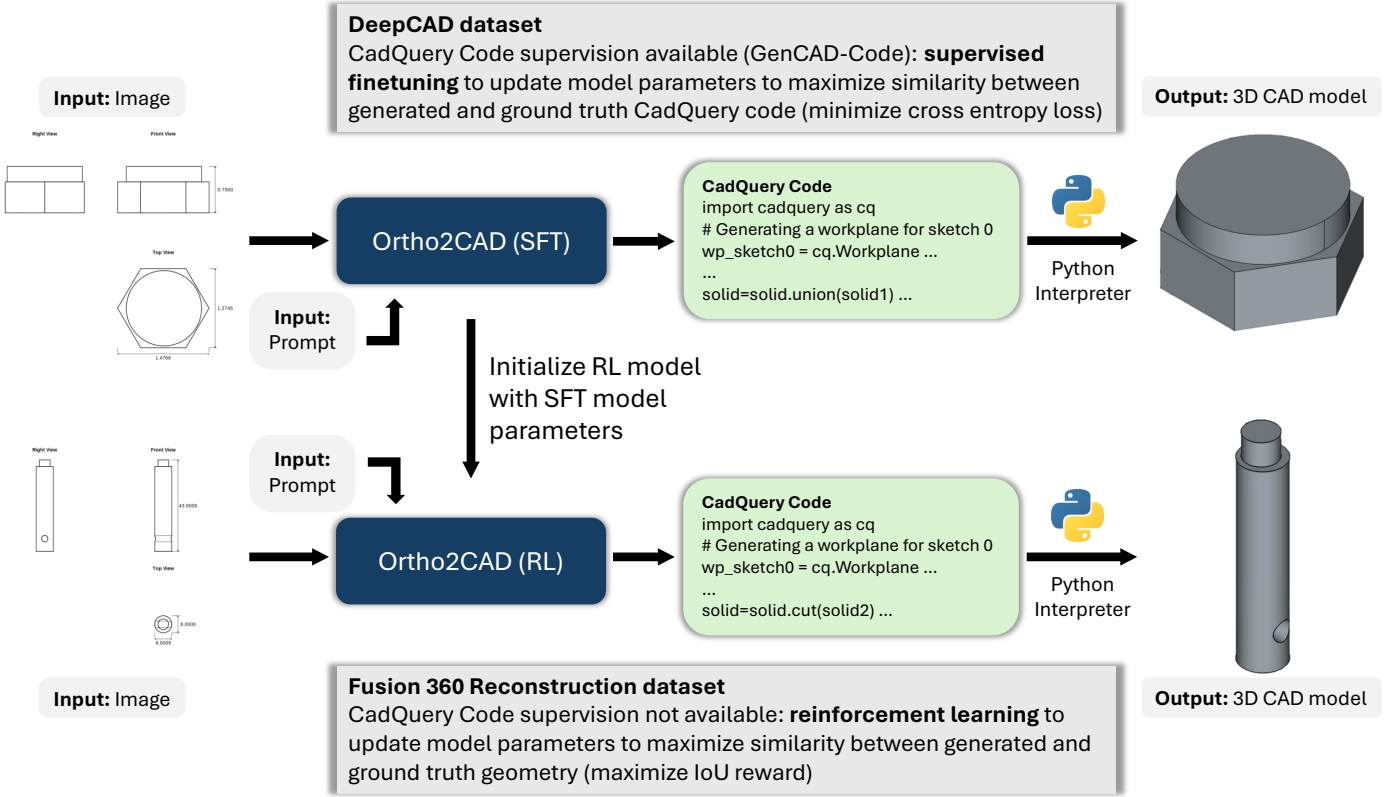


FIGURE 1: Ortho2CAD: Our model takes an orthographic drawing image and a text prompt as input and generates a CadQuery code that compiles into a STEP model. We perform supervised finetuning (SFT) on a pretrained VLM when CadQuery code labels are available; for datasets without supervision, we initialize from the SFT model and apply geometry-grounded reinforcement learning (RL).

geometric and format reward. Govindarajan et al. [20] generate a dataset annotated with high-quality, human-like descriptions and fine-tune code-LLMs to generate CAD sequences represented in a JSON-based format from natural language descriptions. However, all the above approaches do not consider the problem of image to CAD generation.

VLM-based CAD program synthesis. Moving beyond text, VLM-based methods have begun to infer CAD programs from images to support reverse engineering and design-from-visual-input workflows. Alrashedy et al. [21] introduce CAD-CodeVerify to iteratively verify and improve 3D objects generated from CAD code by producing feedback from a VLM and also deviation correction prompts for the VLM. Image2CADSeq [22] creates a pipeline to reverse engineer CAD models by processing images as input and generating CAD sequences. However, it limits itself to a small transformer model and a simplified domain specific language (DSL) and does not utilize pretrained VLMs. Within the CadQuery ecosystem, Doris et al. [3] firstly create the GenCAD-Code dataset and release an open-source VLM (CAD-Coder) trained to produce CadQuery codes from images, demonstrating that modern VLMs can generate editable

CAD. The majority of VLM-based program synthesis work focuses on perspective renders or product images and do not consider engineering drawings as the specification modality.

Reinforcement learning and execution-grounded optimization. Reinforcement learning algorithms such as GRPO [23, 24] and its evolutions [12, 25] have recently gained substantial traction as core post-training methods for LLMs/VLMs. These outcome-grounded approaches move beyond static supervised fine-tuning by optimizing policies against explicit reward signals derived from human preferences or programmable/verifiable objectives, enabling tighter control over generation quality and alignment. Because CAD code must compile and the resulting solid must satisfy geometric constraints, RL has been utilized in recent CAD reconstruction works. Cadrille [4] proposes a multimodal CAD reconstruction framework that uses RL to improve executable program generation with feedback computed from executed CAD outputs, showing that RL can substantially improve reconstruction quality beyond supervised initialization. Relatedly, [26] propose a multimodal chain-of-thought guided RL framework for CAD code generation, showing that reasoning-augmented post-training and

reward shaping can improve executability and geometric accuracy. However, both the above works do not consider the problem of orthographic drawings as input. Also, since we were unable to locate any publicly available implementation of their RL code at the time of writing of this paper, we exclude these methods in our results comparisons. RL has also been investigated specifically for orthographic drawing inputs in [9]. This paper achieves high quality reconstructions of parametric CAD from 2D orthographic drawings using a DQN-style RL agent, but it assumes cleaned, fully-informative three-view inputs and omits common drafting conventions like dashed hidden lines and key dimensions. Because it depends on strict preprocessing and idealized drawing assumptions, the approach is hard to scale to messy real-world drawings. Unlike VLM based frameworks, it does not leverage pretrained VLM generalization or a portable program representation like CadQuery code for broader scalability.

Taken together, the existing literature establishes (i) LLM/VLM code synthesis as a scalable path to editable CAD generation, and (ii) reinforcement learning as a practical mechanism for aligning code generation with geometry-level objectives under weak or missing code supervision. Engineering drawings, specifically multi-view orthographic projections, remain under-represented as an input modality in the recent CadQuery-focused VLM literature, despite being the dominant format for communicating manufacturing-ready design intent. This motivates treating dimensioned orthographic drawings as an input for executable CadQuery code generation. For this we create a dataset generation pipeline for dimensioned orthographic drawings, and leverage supervised fine-tuning where paired programs exist and reinforcement learning where they do not for editable parametric 3D CAD generation.

3 METHOD

We present details of our dataset generation procedure and our supervised finetuning and reinforcement learning frameworks here.

3.1 Dataset Generation

Large-scale datasets of dimensioned, multi-view orthographic engineering drawings aligned with 3D CAD are not readily available, which limits systematic development and evaluation of drawing-to-editable-CAD models. To address this gap, we create a scalable synthetic data generation python code that converts 3D CAD shapes into standardized orthographic drawings using pythonOCC, an open-source Python wrapper for OpenCascade that is straightforward to install and run in a lightweight virtual environment.

Given an input CAD model in STEP format, our python code loads the B-Rep geometry and renders three standard orthographic views (front, top, and right) using first angle projection

and following technical drawing conventions with solid strokes for visible edges, dashed strokes for hidden edges and three key dimensions shown. These dimensions define the bounding box of the part, but do not capture all the geometric features pertinent to the drawing. For example, the inner diameter of a hollow cylinder will not be present and will have to be deciphered from the ratio of the inner diameter and a key dimension available in that direction. However, we believe that including some dimensions, unlike most prior works that do not include any dimensions, is an important step towards perfectly dimensioned drawings, and automation of dimensioning all features in a model is left for future work. Our drawings resemble the raster artifacts used in industrial workflows and preserve geometric fidelity to the original CAD. We generate orthographic drawings with the STEP files in the DeepCAD and Fusion 360 reconstruction datasets. We present a comparison of the datasets in Table 1. Both have only sketch and extrude operations, and training of VLMs with a more diverse set of operations is left for future work. For the DeepCAD dataset, we set a timeout of 10 seconds for generation of a drawing, which serves the purpose of faster generation and also mimicking real scenarios where sometimes certain views may be missing.

The data generation code can be applied to any available dataset that contains STEP files and we publish it on our code repository for this paper.

3.2 Supervised Finetuning

Problem setup. In the supervised regime, our goal is to learn a conditional distribution over executable CadQuery codes given an input query. Let q denote the input query consisting of an orthographic drawing image and an instruction prompt, and let $\tau = (y_1, \dots, y_T)$ denote the target CadQuery token sequence. We fine-tune a pretrained vision-language model (VLM) π_θ to maximize the likelihood of the ground-truth CadQuery code from the GenCAD-Code dataset. We use Qwen3-VL-8B-Instruct [11] as the pretrained backbone due to its reasonable parameter count relative to the CAD-Coder baseline ($\sim 13B$ parameters) while still providing a strong multimodal foundation.

Input/output formatting. Each training sample consists of: (i) a three-view dimensioned orthographic drawing I (PNG raster), and (ii) a fixed natural-language instruction prompt “Generate the CADQuery code needed to create the CAD for the provided image. Just the code, no other words.” which matches the prompt template used by CAD-Coder. While this prompt can be improved, we choose to use this for simplicity and plan to improve on this prompt in future work. For off-the-shelf models where finetuning is not performed, we used a similar strategy as [3] and appended to the prompt: “Assign the final solid to the variable ‘solid’ in the last line of code. Do not export or visualize the solid.” to match the evaluation of generated code. The su-

TABLE 1: Comparison of DeepCAD and Fusion 360 Reconstruction datasets.

Criterion	DeepCAD (with GenCAD-Code)	Fusion 360 Reconstruction
Number of Models	GenCAD-Code train/test/val split of 147289/7355/9027; 100 test subset for final evaluation same as [3]	Official train/test split of 6900/1725; 100 test subset for final evaluation
Ground truth CadQuery code available; usable for SFT	Yes, used in this work	No
Ground truth STEP available; usable for RL	Yes, but not explored in this work	Yes, used in this work

pervision target is the corresponding CadQuery code τ from the GenCAD-Code dataset, which can be executed to produce a 3D CAD solid.

Objective. We minimize the standard autoregressive cross-entropy (negative log-likelihood) loss over the CadQuery token sequence conditioned on the query $q = (I, \text{prompt})$:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(q, \tau) \sim \mathcal{D}} \left[\sum_{t=1}^T \log \pi_{\theta}(y_t \mid q, y_{<t}) \right] \quad (1)$$

where $\mathcal{D}$ is our dataset of generated orthographic drawings from the DeepCAD dataset and corresponding CadQuery code pairs from the GenCAD-Code dataset. This objective directly encourages token-level correctness of the generated CadQuery code.

Training details. We fine-tune Qwen3VL-8B-Instruct for 5 epochs on the DeepCAD-derived orthographic drawing dataset paired with GenCAD-Code CadQuery codes, using the train/validation/test splits from [3]. We follow the standard recommended configuration for Qwen3VL for finetuning: the vision model is kept fixed, and the multimodal MLP/projection layers that map vision features into the LLM token space and the language backbone (LLM) are kept trainable. We use a learning rate of $1e-5$, per-device batch size of 4, and 4 gradient accumulation steps, which we found to provide stable optimization in this setting. The training takes 11 hours on 4 H100 GPUs. For the CAD-Coder baseline, we use their recommended settings, and the training on our DeepCAD-derived orthographic drawing dataset paired with GenCAD-Code CadQuery codes also takes 11 hours on 4 H100 GPUs.

Evaluation metric and details. Following [3], we evaluate geometric reconstruction accuracy using the intersection-over-union (IoU) between the solid produced by executing the generated CadQuery code (exported to STEP) and the ground-truth CAD solid (ground-truth STEP). For model outputs that cannot

be executed, we assign them $\text{IoU} = 0$. We use the same evaluation subset of 100 representative examples that is used in [3] for consistency and comparability.

3.3 Reinforcement Learning

Motivation: learning without code supervision. Many practical CAD reconstruction datasets provide ground-truth geometry (e.g., STEP) but do not provide aligned, standardized CadQuery codes for supervised code synthesis. To train in this setting, we use execution-grounded reinforcement learning (RL): the VLM generates CadQuery code and the code is executed to produce a CAD solid. Since code execution and solid generation are not differentiable operations, RL provides a natural mechanism to improve the policy (VLM) using scalar feedback computed from the executed geometry.

Reward from executable geometry. For a query $q = (I, \text{prompt})$, let τ denote a generated CadQuery code and $S(\tau)$ denote the solid obtained by executing τ (exported to a STEP file for IoU computation). Let $S^*(q)$ denote the ground-truth solid (STEP file). We define the reward as the solid intersection-over-union:

$$R(q, \tau) = \text{IoU}(S(\tau), S^*(q)) \quad (2)$$

where IoU is defined as in [3]. In Figure 2, we show visualization of the IoU for two parts. If the generated code is invalid (parse/runtime failure or no valid solid), we set $R(q, \tau) = 0$ and do not apply any additional penalty term.

Why Dr. GRPO with sequence-level optimization We draw on the design principles of Dr. GRPO [12]. It analyzes how group-based reinforcement learning objectives can introduce length and normalization induced biases and proposes simple modifications that yield more reliable optimization dynamics when learning is driven by sparse, terminal outcome rewards.

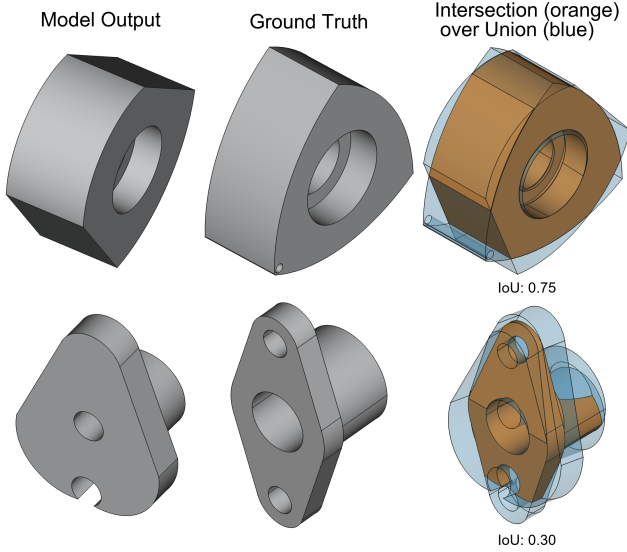


FIGURE 2: Visualization of IoU. The model output and ground truth are normalized and aligned [3] for calculating the IoU.

This closely matches our setting: the reward is only observed after executing a complete CadQuery program, so overly long or unstable generations increase execution-failure rates and effectively censor the learning signal, making it important to avoid update rules that implicitly favor longer (but not better) outputs. Concretely, we adopt mean-centered group advantages without additional length or variance normalization, and we follow the GSPO perspective [27] that when rewards are assigned at the sequence level, optimization should also be performed at the sequence level (via full-trajectory log-likelihoods) rather than via token-level credit heuristics, which improves stability for long completions and better matches the granularity of executable-geometry rewards.

Group sampling and advantages. At each iteration, for each query q we sample a group of G candidate codes $\{\tau^{(g)}\}_{g=1}^G$ from the current policy $\pi_\theta(\cdot | q)$ and compute rewards $R(q, \tau^{(g)})$ using Eq. 2. We compute a mean-centered group advantage (with no additional reward scaling):

$$A^{(g)}(q) = R(q, \tau^{(g)}) - \frac{1}{G} \sum_{j=1}^G R(q, \tau^{(j)}). \quad (3)$$

This produces a purely relative learning signal: for a given drawing, above-average samples are reinforced and below-average samples are suppressed. We perform one gradient update using each sampled group and then discard it; new groups are sampled from the updated policy in the next iteration.

Autoregressive sequence likelihood. An autoregressive model factorizes the probability of a code $\tau^{(g)} = (y_1^{(g)}, \dots, y_{T^{(g)}}^{(g)})$ into a product of conditional token probabilities:

$$\pi_\theta(\tau^{(g)} | q) = \prod_{t=1}^{T^{(g)}} \pi_\theta(y_t^{(g)} | q, y_{<t}^{(g)}). \quad (4)$$

Taking the logarithm converts this product into a sum:

$$\log \pi_\theta(\tau^{(g)} | q) = \sum_{t=1}^{T^{(g)}} \log \pi_\theta(y_t^{(g)} | q, y_{<t}^{(g)}). \quad (5)$$

Policy optimization objective. We update the policy using a sequence-level objective that increases the likelihood of sampled codes with higher relative reward within each group:

$$\mathcal{L}_{\text{RL}}(\theta) = -\mathbb{E}_{q \sim \mathcal{D}} \left[\frac{1}{LG} \sum_{g=1}^G \log \pi_\theta(\tau^{(g)} | q) A^{(g)}(q) \right] \quad (6)$$

where L is a fixed global scaling constant: the maximum completion length. Intuitively, Eq. 6 increases the probability of complete CadQuery codes that achieve higher IoU than other samples drawn for the same input drawing, without requiring any ground-truth CadQuery codes.

Training and evaluation details. We use the finetuned VLM from section 3.2 and train on the Fusion 360 reconstruction dataset using the dataset provided train/test split [6]. We keep the vision model fixed during training, and the multimodal MLP/projection layers and the language backbone (LLM) are kept trainable. We use the same prompt format as in SFT for the respective models. Each training sample provides an orthographic drawing (generated from the ground-truth STEP via our pipeline) and the ground-truth STEP geometry used to compute the IoU reward. No ground-truth CadQuery codes are used during RL. We provide a summary of the hyperparameters used in the RL training in Table 2. A maximum model length of 8192 similar to Qwen3VL official settings is used in SFT, and thus we use a maximum completion length (L) of 7680 tokens in RL (we obtain this by subtracting from 8192 the input tokens count and a small buffer for change in input). For further details on hyperparameters, please refer to our code available at <https://github.com/AdityaJoglekar/Ortho2CAD>. The training takes 100 hours on 4 H100 GPUs. As in supervised finetuning evaluation, we report IoU on a subset of 100 test examples due to the high computational cost and time of generating and evaluating the output of all VLMs.

TABLE 2: Reinforcement learning hyperparameters

Hyperparameter	Value
Number of Epochs	2
Learning Rate	1e-6
Effective batch size	64
Number of generations (G) per query q	8

4 RESULTS

This section presents quantitative and qualitative evaluation of the proposed approach across two training regimes: supervised fine-tuning on DeepCAD with ground-truth CadQuery codes, and reinforcement learning on Fusion 360 without code supervision.

For the baselines, we consider three families that collectively span (i) strong open-source pretrained vision-language models, (ii) a strong closed-source general-purpose model representative of the best off-the-shelf capability at the time of writing, and (iii) a domain-specialized state-of-the-art model trained specifically for image-to-CadQuery generation:

1. **Qwen3VL family (open-source VLMs).** We evaluate pretrained models from the Qwen3VL family to establish a strong, reproducible open-source reference point for multi-modal CadQuery code synthesis. These models are attractive because they provide modern VLM capabilities (image understanding + code generation) while remaining directly finetunable and deployable, making them a practical baseline for follow-on supervised and RL-based adaptation.
2. **OpenAI GPT 5.2 (closed-source, general-purpose VLM).** We include GPT 5.2 [28] as a representative high-capability closed-source model which at the time of writing is OpenAI’s latest standard model. This baseline serves the purpose of providing an estimate of how far one can get without task-specific training, and it contextualizes the open-source results against a strong general-purpose system widely used as a reference point in practice.
3. **CAD-Coder (domain-specific state of the art).** CAD-Coder [3] is included because it is the state-of-the-art open-source model for image to CadQuery code generation and is trained explicitly for executable CadQuery synthesis. As such, it provides the most relevant specialized comparison point for our setting, allowing us to measure whether conditioning on orthographic drawings and using a stronger backbone with targeted post-training yields improvements over the best existing CadQuery-focused VLM baseline.

We report (i) *Valid Codes*, the fraction of generations that are

TABLE 3: A quantitative comparison of our model against different baselines on the DeepCAD dataset benchmark.

Model	Valid Codes	IoU
Qwen3VL8B Instruct	70%	0.3747
Qwen3VL32B Instruct	76%	0.4919
GPT 5.2	90%	0.5987
CAD-Coder	100%	0.7361
Ortho2CAD (ours)	100%	0.7922

syntactically valid and execute to a solid, and (ii) *IoU*, the mean intersection-over-union between the solid produced by executing generated CadQuery code and the ground-truth STEP geometry, computed on the evaluation examples.

4.1 Supervised Fine-tuning Results on DeepCAD

We evaluate the supervised fine-tuning approach on the DeepCAD dataset with the orthographic drawings paired with ground-truth CadQuery codes from GenCAD-Code [3], in Table 3, which presents quantitative results comparing our Ortho2CAD model against several baselines.

Key findings from supervised fine-tuning.

Code validity. Ortho2CAD achieves 100% valid code generation on the test subset, matching the CAD-Coder baseline. This indicates that the model successfully learns to produce syntactically correct and executable CadQuery codes from orthographic drawings. In contrast, GPT-5.2 without domain-specific fine-tuning achieves only 90% validity, highlighting the importance of specialized training for this task.

Geometric reconstruction quality. Ortho2CAD achieves a mean IoU of 0.7922, representing a 7.6% relative improvement over the best baseline model (CAD-Coder). We show the IoU distribution achieved with our model on the test set in Figure 4. Overall, the results demonstrate that (i) orthographic drawings provide effective conditioning signals for parametric CAD synthesis, and (ii) fine-tuning a stronger, more recent VLM backbone (Qwen3-VL-8B-Instruct) on the same task yields measurable improvements in geometric accuracy compared to the CAD-Coder architecture which is based on finetuning the Llava architecture [29].

The off-the-shelf GPT-5.2 model, despite being a very large and capable foundation model, achieves only 0.5905 IoU without

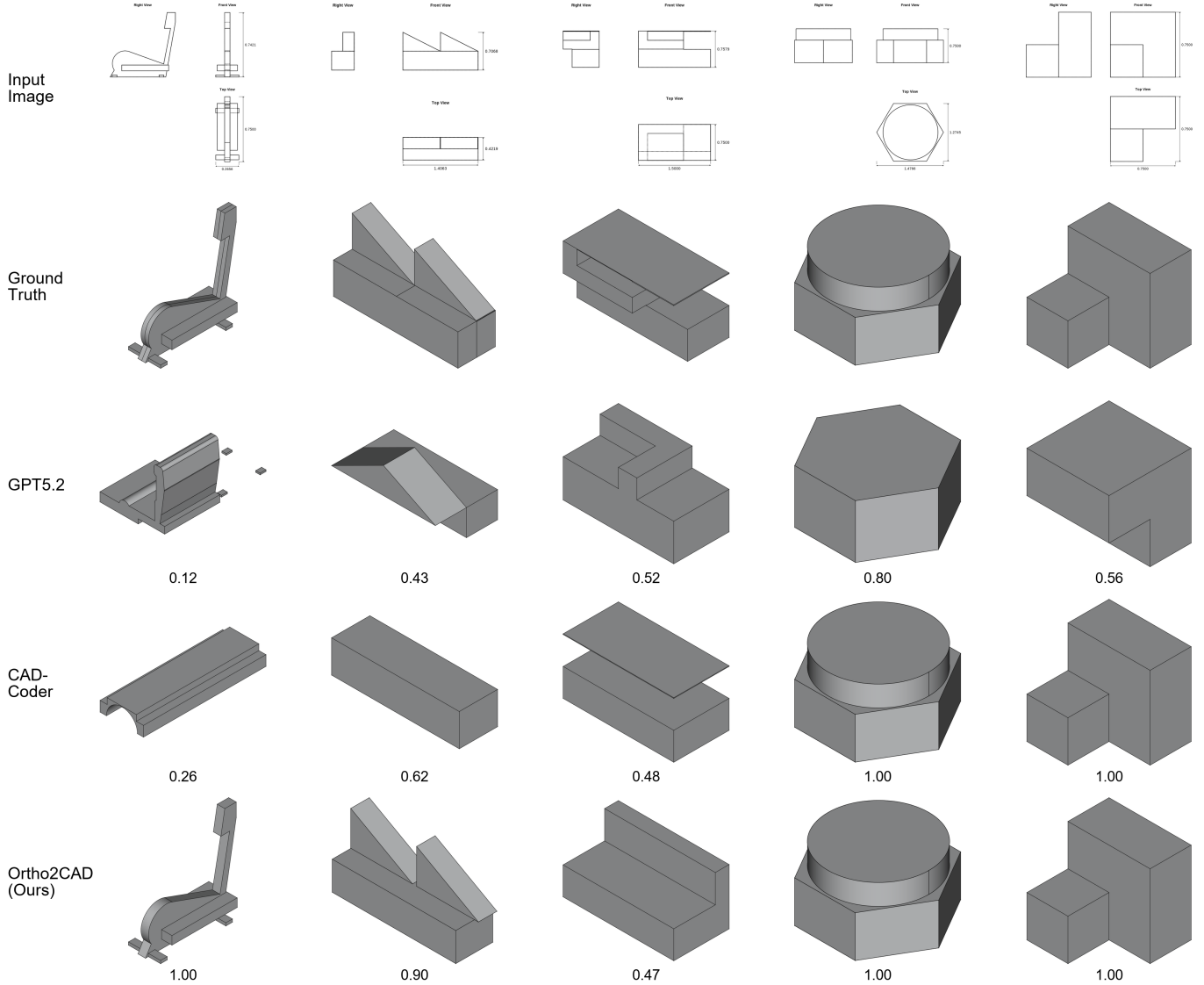


FIGURE 3: A qualitative comparison of our model against the baselines on five test subset examples from the DeepCAD dataset. The IoU of the generated CAD models is shown in their respective captions.

specialized fine-tuning, underscoring the domain-specific nature of the drawing-to-CAD task and the necessity of targeted training on engineering drawings and CAD code generation.

Qualitative analysis. Figure 3 presents a qualitative comparison across five representative test examples from DeepCAD, showing the input orthographic drawing, ground-truth CAD solid, and generated solids from GPT-5.2, CAD-Coder, and Ortho2CAD. Visual inspection reveals that Ortho2CAD more accurately captures complex geometric features. Examples where all models perform well ($\text{IoU} > 0.85$) typically involve simple prismatic parts with few features. Examples where Ortho2CAD out-

performs baselines often involve parts with multiple interacting features and more complex topology requiring careful sequencing of CAD operations. In Figure 7 (Appendix), we evaluate the success and failure cases: the best two and worst two results in the DeepCAD test subset using our model are shown to gain a better understanding of the model’s performance.

4.2 Reinforcement Learning Results on Fusion 360

Table 4 presents results on the Fusion 360 reconstruction dataset, which lacks ground-truth CadQuery codes and thus motivates the reinforcement learning approach. We compare against

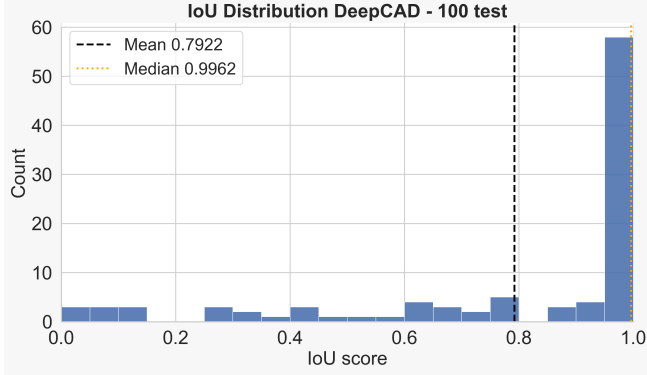


FIGURE 4: The distribution of IoU scores of reconstructions obtained with our Ortho2CAD model on the DeepCAD test subset (100 examples).

TABLE 4: A quantitative comparison of our model against different baselines on the Fusion 360 dataset benchmark.

Model	Valid Codes	IoU
Qwen3VL8B Instruct	58%	0.2730
Qwen3VL32B Instruct	66%	0.3600
GPT 5.2	78%	0.5181
CAD-Coder DeepCAD SFT	90%	0.2473
Ortho2CAD DeepCAD SFT (ours)	96%	0.3697
Ortho2CAD RL (ours)	100%	0.5601

the same baselines, and report results for our model after supervised fine-tuning on DeepCAD and after RL training with geometric feedback on the Fusion 360 dataset.

We can see that the same off-the-shelf baselines (Qwen models and GPT 5.2) perform worse on the Fusion 360 reconstruction test subset than they did on the DeepCAD test subset, indicating that the Fusion 360 test subset has more complex parts that are harder to reconstruct.

The Ortho2CAD model trained purely via supervised fine-tuning (SFT) on DeepCAD achieves 96% code validity when transferred to Fusion 360, but only 0.3697 mean IoU. The fact that code validity remains high (96%) while geometric accuracy drops sharply suggests that the model has learned the CadQuery syntax and can generate executable codes, but the geometric content of those codes does not align well with the target dataset distribution without additional adaptation. For reinforcement learn-

ing, the ability to generate executable codes is very important for fast and stable training and this model acts as our starting point for RL training. Our RL reward of IoU is scale invariant and the dimensions units of the output models can be scaled as per user requirement.

RL-driven improvement. After RL training on the Ortho2CAD DeepCAD SFT model, the model achieves 100% code validity and 0.5601 mean IoU on the Fusion 360 reconstruction dataset. This represents:

1. A 8.1% relative improvement in IoU over the best baseline model (GPT-5.2).
2. Demonstration that executable-geometry feedback can effectively adapt a model to new data distributions without requiring ground-truth CadQuery codes.

We show the IoU distribution achieved with our model on the test set in Figure 6. We also present a qualitative comparison between our RL model and the next best performing model in terms of IoU (GPT 5.2) across five representative test examples from the Fusion 360 dataset in Figure 5. We can observe that some parts are more complicated than in the DeepCAD dataset, while the last part (a cylinder) is comparatively easy to reconstruct. GPT 5.2 does not output valid CadQuery codes for two of the examples. It outperforms our model on example 2, but lags behind in the other two examples it outputs valid code for. In Figure 8 (Appendix), we evaluate the success and failure cases for a better understanding of our Ortho2CAD RL model. We can observe that similar to the DeepCAD dataset, thin complex geometries result in low reconstruction IoU.

5 CONCLUSION, LIMITATIONS AND FUTURE WORK

The combined results demonstrate three key findings:

1. Parametric CAD synthesis using VLMs with orthographic drawings as inputs is possible, with finetuning of modern VLM models on CadQuery code giving excellent results.
2. Reinforcement learning enables learning on geometric feedback, and without CadQuery code supervision, with our model surpassing all baselines in code validity and IoU results.
3. Orthographic drawing dataset generation python code enables scalable research for different CAD datasets.

The current results leave a significant headroom for improvement in terms of IoU on complex datasets where code supervision is unavailable, and which may involve operations not seen in the DeepCAD CadQuery code dataset (for example the ABC dataset [10]). We also observe that GPT 5.2 gives cleaner output code with comments and better intermediate variable names although this does not translate to better code validity and geometry as we see from the results. However, a better and more

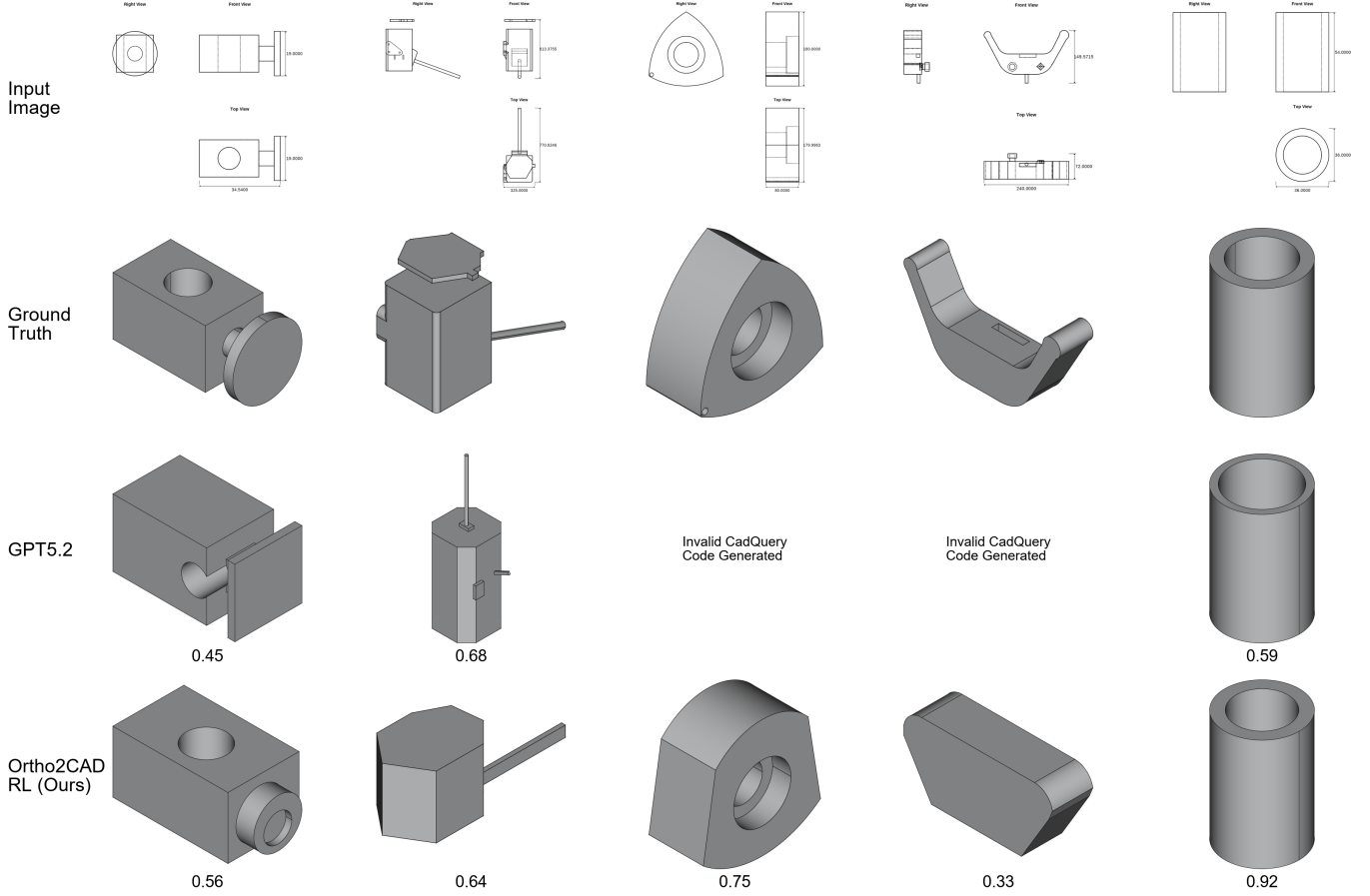


FIGURE 5: A qualitative comparison of our Ortho2CAD RL model against GPT 5.2 on five test subset examples from the Fusion 360 dataset. The IoU of the generated CAD models is shown in their respective captions. Here, we only include our results (best) against GPT5.2 (second best) for brevity. See Table 4 for reference.

diverse CadQuery code database for SFT that includes a variety of operations and is less noisy will be very beneficial. Exploring SFT with a combination of the GenCAD-Code [3], CAD-Recode [30] and any other available CadQuery datasets is left for future work. For better reconstruction accuracy, reinforcement learning training could be improved with richer reward signals such as, in addition to IoU reward, rewarding the similarities in the input drawing and orthographic projection of the output 3D CAD. A great direction for further research is adding chain-of-thought (CoT) reasoning to the models, which may improve the RL signals by reducing exploration variance by providing structured intermediate outputs that guide subsequent code. Creation of high quality CoT data will be the key to progress here.

Additionally, engineering design is iterative: designers generate initial models, evaluate them, identify errors, and refine. Current inference generates a single program and stops. Work on inference time scaling for CAD [21] demonstrates that iterative refinement with automatic feedback can improve reconstruction

quality. An important research problem is to find the right balance between using large closed-source models that can have strong reasoning and feedback-integration capabilities but are expensive to operate with and may not give great first-pass results and using fine-tuned open-source models that can generate high-quality first-pass results but may struggle with iterative refinement without additional targeted training. Future work could involve a hybrid approach that combines the two for optimum results.

ACKNOWLEDGMENT

This work was supported by the MiSUMi Corporation. The research was conducted on ORCHARD, a high-performance cloud computing cluster made available by Carnegie Mellon University. FreeCAD software was used for visualizing the 3D parts in all the figures. We thank Faez Ahmed, Joe Joseph, Soji Yamakawa and Vedant Puri for insightful discussions.

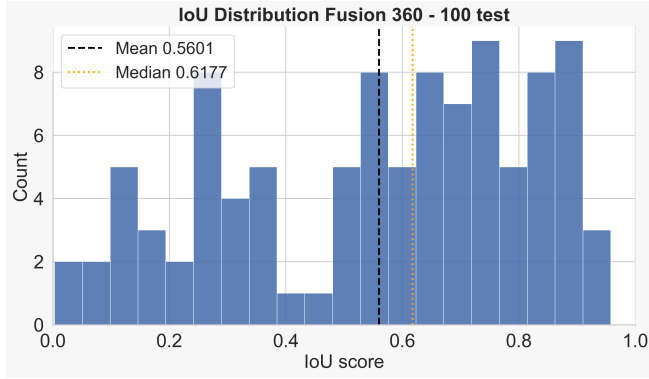


FIGURE 6: The distribution of IoU scores of reconstructions obtained with our Ortho2CAD RL model on the Fusion 360 reconstruction test subset (100 examples).

REFERENCES

- [1] Zhang, W., Joseph, J., Yin, Y., Xie, L., Furuhashi, T., Yamakawa, S., Shimada, K., and Kara, L. B., 2023. “Component segmentation of engineering drawings using graph convolutional networks”. *Computers in Industry*, **147**, p. 103885.
- [2] Guan, Y., Wang, X., Xing, X., Zhang, J., Xu, D., and Yu, Q., 2025. “Cad-coder: Text-to-cad generation with chain-of-thought and geometric reward”. *arXiv preprint arXiv:2505.19713*.
- [3] Doris, A. C., Alam, M. F., Heyrani Nobari, A., and Ahmed, F., 2025. “Cad-coder: An open-source vision-language model for computer-aided design code generation”. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, Vol. 89220, American Society of Mechanical Engineers, p. V03AT03A031.
- [4] Kolodiaznyy, M., Tarasov, D., Zhemchuzhnikov, D., Nikulin, A., Zisman, I., Vorontsova, A., Konushin, A., Kurenkov, V., and Rukhovich, D., 2025. “cadrille: Multi-modal cad reconstruction with online reinforcement learning”. *arXiv preprint arXiv:2505.22914*.
- [5] Wu, R., Xiao, C., and Zheng, C., 2021. “Deepcad: A deep generative network for computer-aided design models”. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 6772–6782.
- [6] Willis, K. D., Pu, Y., Luo, J., Chu, H., Du, T., Lambourne, J. G., Solar-Lezama, A., and Matusik, W., 2021. “Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences”. *ACM Transactions on Graphics (TOG)*, **40**(4), pp. 1–24.
- [7] Zhang, C., Piquié, R., Polette, A., Carasi, G., De Charnace, H., and Pernot, J.-P., 2023. “Automatic 3d cad models reconstruction from 2d orthographic drawings”. *Computers & Graphics*, **114**, pp. 179–189.
- [8] FreeCAD Team, 2026. FreeCAD. Accessed: 2026-02-20.
- [9] Zhang, C., Polette, A., Piquié, R., Iida, M., De Charnace, H., and Pernot, J.-P., 2025. “Reinforcement learning-based parametric cad models reconstruction from 2d orthographic drawings”. *Computer-Aided Design*, p. 103925.
- [10] Koch, S., Matveev, A., Jiang, Z., Williams, F., Artemov, A., Burnaev, E., Alexa, M., Zorin, D., and Panozzo, D., 2019. “Abc: A big cad model dataset for geometric deep learning”. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 9601–9611.
- [11] Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., et al., 2025. “Qwen3-vl technical report”. *arXiv preprint arXiv:2511.21631*.
- [12] Liu, Z., Chen, C., Li, W., Qi, P., Pang, T., Du, C., Lee, W. S., and Lin, M., 2025. “Understanding r1-zero-like training: A critical perspective”. *arXiv preprint arXiv:2503.20783*.
- [13] Nichol, A., Jun, H., Dhariwal, P., Mishkin, P., and Chen, M., 2022. “Point-e: a system for generating 3d point clouds from complex prompts (2022)”. *arXiv preprint arXiv:2212.08751*.
- [14] Poole, B., Jain, A., Barron, J. T., and Mildenhall, B., 2022. “Dreamfusion: Text-to-3d using 2d diffusion”. *arXiv preprint arXiv:2209.14988*.
- [15] Siddiqui, Y., Alliegro, A., Artemov, A., Tommasi, T., Sirigatti, D., Rosov, V., Dai, A., and Nießner, M., 2024. “Meshgpt: Generating triangle meshes with decoder-only transformers”. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 19615–19625.
- [16] Xiang, J., Lv, Z., Xu, S., Deng, Y., Wang, R., Zhang, B., Chen, D., Tong, X., and Yang, J., 2025. “Structured 3d latents for scalable and versatile 3d generation”. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 21469–21480.
- [17] Xu, X., Lambourne, J., Jayaraman, P., Wang, Z., Willis, K., and Furukawa, Y., 2024. “BrepGen: A b-rep generative diffusion model with structured latent geometry”. *ACM Transactions on Graphics (TOG)*, **43**(4), pp. 1–14.
- [18] Liu, Y., Xu, D., Yu, X., Xu, X., Cohen-Or, D., Zhang, H., and Huang, H., 2025. “HOLA: B-rep generation using a holistic latent representation”. *ACM Transactions on Graphics (TOG)*, **44**(4), pp. 1–25.
- [19] Khan, M. S., Sinha, S., Sheikh, T. U., Stricker, D., Ali, S. A., and Afzal, M. Z., 2024. “Text2cad: Generating sequential cad designs from beginner-to-expert level text prompts”. *Advances in Neural Information Processing Systems*, **37**, pp. 7552–7579.
- [20] Govindarajan, P., Baldelli, D., Pathak, J., Fournier, Q., and Chandar, S., 2025. “Cadmium: Fine-tuning code language models for text-driven sequential cad design”. *arXiv*

preprint arXiv:2507.09792.

- [21] Alrashedy, K., Tambwekar, P., Zaidi, Z., Langwasser, M., Xu, W., and Gombolay, M., 2024. “Generating cad code with vision-language models for 3d designs”. *arXiv preprint arXiv:2410.05340*.
- [22] Li, X., and Sha, Z., 2025. “Image2cadseq: Computer-aided design sequence and knowledge inference from product images”. In International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Vol. 89213, American Society of Mechanical Engineers, p. V02BT02A025.
- [23] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al., 2024. “Deepseekmath: Pushing the limits of mathematical reasoning in open language models”. *arXiv preprint arXiv:2402.03300*.
- [24] Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., et al., 2025. “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning”. *arXiv preprint arXiv:2501.12948*.
- [25] Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Dai, W., Fan, T., Liu, G., Liu, L., et al., 2025. “Dapo: An open-source llm reinforcement learning system at scale”. *arXiv preprint arXiv:2503.14476*.
- [26] Niu, K., Yu, H., Chen, Z., Zhao, M., Fu, T., Li, B., and Xue, X., 2025. “From intent to execution: Multimodal chain-of-thought reinforcement learning for precise cad code generation”. *arXiv preprint arXiv:2508.10118*.
- [27] Zheng, C., Liu, S., Li, M., Chen, X.-H., Yu, B., Gao, C., Dang, K., Liu, Y., Men, R., Yang, A., et al., 2025. “Group sequence policy optimization”. *arXiv preprint arXiv:2507.18071*.
- [28] OpenAI, 2025. Introducing GPT-5.2. Accessed: 2026-02-20.
- [29] Liu, H., Li, C., Wu, Q., and Lee, Y. J., 2023. “Visual instruction tuning”. *Advances in neural information processing systems*, **36**, pp. 34892–34916.
- [30] Rukhovich, D., Dupont, E., Mallis, D., Cherenkova, K., Kacem, A., and Aouada, D., 2025. “Cad-recode: Reverse engineering cad code from point clouds”. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 9801–9811.

A APPENDIX

We evaluate the success and failure cases for a better understanding of our Ortho2CAD model on the DeepCAD 100 test subset in Figure 7 and our Ortho2CAD RL model on the Fusion 360 Reconstruction 100 test subset in Figure 8.

B LLM Usage

LLMs assisted with grammar and readability changes and with boilerplate code in our experimental scripts.

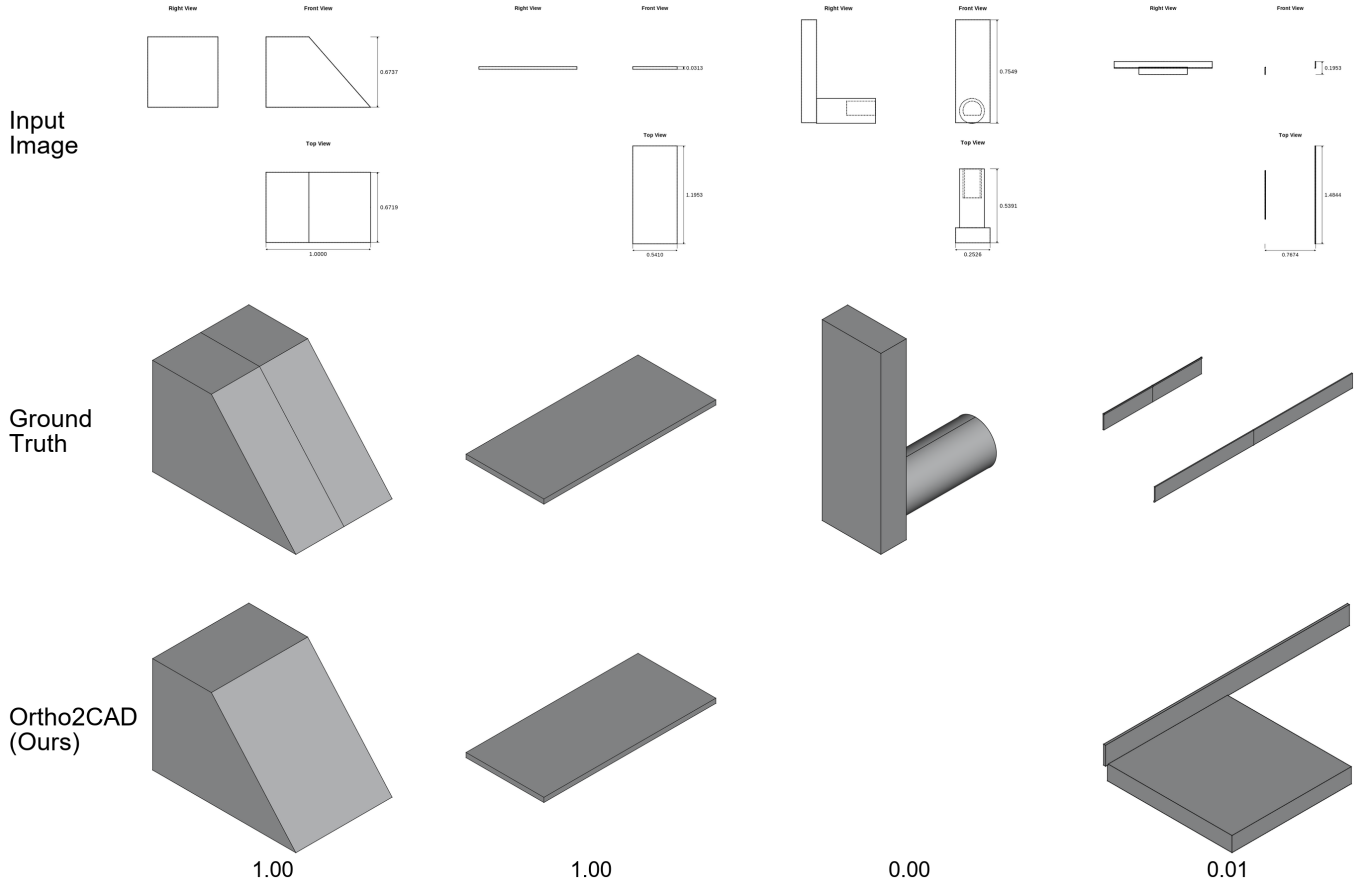


FIGURE 7: The best two and worst two results in the DeepCAD test subset using our model are shown here to gain a better understanding of the model's performance (IoU in respective captions). We observe that for the worst reconstruction (3rd column in the figure, with the hollow cylinder attached to the rectangular block), the cadquery code was valid and produced a valid step file, but the operations do not produce an enclosed volume. For the second worst case, the input image is very hard to decipher due to the thin sheets that are present in the ground truth cad model, and our model faces difficulties in reconstruction.

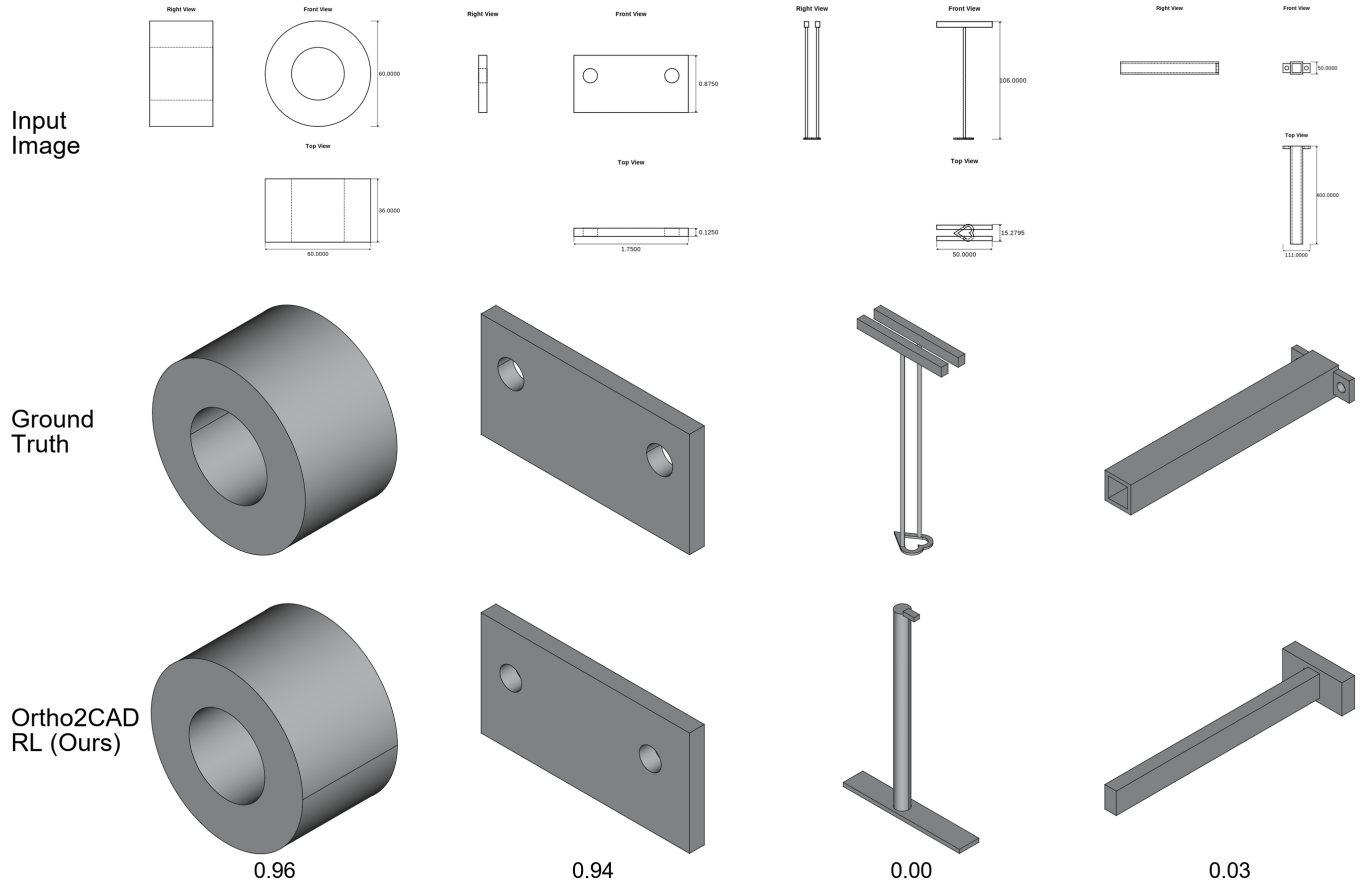


FIGURE 8: The best two and worst two results in the Fusion 360 test subset using our model are shown here to gain a better understanding of the model's performance (IoU in respective captions). We can observe that thin complex geometries result in low reconstruction IoU.